

Here's a refined, enterprise-level prompt you can give to Laravel Copilot, Cursor, Kimi, Claude Code, or any AI coding assistant:

Laravel 13 Multi-Tenant Flash Sale E-Commerce SaaS (API-Only)

Act as a Principal Laravel Architect and Senior Software Engineer.

Build a production-ready, enterprise-grade **Multi-Tenant Flash Sale E-Commerce SaaS API** using the latest Laravel 13 best practices.

Core Requirements

The application must support:

- Multi-tenancy using Stancl Tenancy
 - Subdomain-based tenancy
 - Custom domain tenancy
 - API-only architecture
 - Flash sale campaigns
 - Time-limited product drops
 - Real-time inventory countdown
 - Waitlists for sold-out products
 - Queue-based checkout for high-demand sales
 - Push notifications
 - Subscription billing
 - Tenant onboarding
 - Analytics and reporting
 - Multi-role access control
 - Activity auditing
 - Media management
-

Technical Stack

Implement using:

- Laravel 13
- PHP 8.4+
- MySQL
- Stancl Tenancy
- Laravel Sanctum

- [Laravel Cashier Stripe](#)
 - [Laravel Cashier Paystack](#)
 - [Spatie Permission](#)
 - [Spatie Activitylog](#)
 - [Spatie Media Library](#)
 - [Laravel Queues](#)
 - [Laravel Events & Listeners](#)
 - [Laravel Notifications](#)
 - [Laravel API Resources](#)
-

Architecture Standards

Follow strict Domain-Driven and Service-Oriented Architecture.

Rules

- Controllers must remain thin
 - Business logic must exist only inside Service Classes
 - No business logic inside controllers
 - No business logic inside models
 - No dynamic implementations
 - No repository pattern unless absolutely necessary
 - Use Form Requests for all validations
 - Use API Resources for all responses
 - Use database transactions where necessary
 - Use enums where applicable
 - Use PHPDoc blocks everywhere
 - Use strict typing
 - Use constructor dependency injection
 - Follow SOLID principles
 - Follow PSR-12 standards
 - Use Laravel Actions only when appropriate
 - Use Events and Listeners for side effects
 - Use Policies for authorization
 - Use Service Classes for all business operations
-

Folder Structure

Generate code using the following structure:

app/



```
|— Http/
|   |— Controllers/
|   |   |— Central/
|   |   └─ Tenant/
|   |
|   |— Requests/
|   |   |— Central/
|   |   └─ Tenant/
|   |
|   └─ Resources/
|       |— Central/
|       └─ Tenant/
|
|— Models/
|   |— Central/
|   └─ Tenant/
|
|— Services/
|   |— Central/
|   └─ Tenant/
|
|— Enums/
|   |— Central/
|   └─ Tenant/
|
|— Events/
|   |— Central/
|   └─ Tenant/
|
|— Listeners/
|   |— Central/
|   └─ Tenant/
|
|— Notifications/
|   |— Central/
|   └─ Tenant/
|
|— Policies/
|   |— Central/
|   └─ Tenant/
|
|— Traits/
|
|— Helpers/
|
└─ Exceptions/
```

Central Application Modules

The central application manages the SaaS platform itself.

Tenant Management

Create:

- Tenant
- Domain
- Tenant Settings

Features:

- Create Tenant
- Update Tenant
- Suspend Tenant
- Activate Tenant
- Delete Tenant
- Tenant Statistics
- Domain Management
- Custom Domain Verification

Services:

⌕ PHP



```
TenantService  
TenantProvisioningService  
DomainService  
TenantSettingsService
```

Subscription & Billing

Implement:

Stripe

Using Laravel Cashier Stripe.

Paystack

Using Laravel Cashier Paystack.

Features:

- Plans

- Subscriptions
- Trials
- Upgrades
- Downgrades
- Cancellations
- Webhooks
- Invoices
- Payment History

Services:

⌕ PHP



```
SubscriptionService
PaymentService
InvoiceService
WebhookService
```

Central User Management

Models:

⌕ PHP



```
CentralUser
Role
Permission
```

Implement:

- Super Admin
- Platform Admin
- Support Agent
- Billing Manager

Using Spatie Permission.

Tenant Application Modules

Each tenant operates as an independent flash-sale store.

Tenant Users

Models:

</> PHP



```
TenantUser  
Customer
```

Roles:

```
Store Owner  
Store Manager  
Sales Manager  
Inventory Manager  
Customer Support  
Customer
```



Use Spatie Permission.

Product Module

Models:

</> PHP



```
Product  
ProductVariant  
Category  
Brand  
Tag  
Attribute  
AttributeValue  
ProductImage  
Inventory
```

Features:

- CRUD
- Variant Management
- Inventory Management
- Media Uploads
- SEO Fields
- Product Visibility

Use Spatie Media Library.

Services:

⌵ PHP



```
ProductService
InventoryService
CategoryService
BrandService
```

Flash Sale Module

Models:

⌵ PHP



```
FlashSale
FlashSaleProduct
FlashSaleRule
```

Features:

- Sale Scheduling
- Start Date
- End Date
- Sale Countdown
- Scheduled Activation
- Automatic Expiration
- Limited Stock Drops
- Limited-Time Offers

Services:

⌵ PHP



```
FlashSaleService
DropSchedulerService
```

Queue-Based Checkout Module

Models:

⌵ PHP



```
CheckoutQueue
CheckoutSession
```

Features:

- Virtual Waiting Room
- Queue Position Tracking
- Session Expiration
- High Traffic Protection
- Fair Checkout Allocation

Services:

</> PHP



QueueService

CheckoutSessionService

Waitlist Module

Models:

</> PHP



Waitlist

WaitlistSubscriber

Features:

- Join Waitlist
- Leave Waitlist
- Back-In-Stock Alerts
- Flash Sale Alerts

Services:

</> PHP



WaitlistService

Cart Module

Models:

</> PHP




```
Cart
CartItem
```

Services:

```
</> PHP
```



```
CartService
```

Order Module

Models:

```
</> PHP
```



```
Order
OrderItem
OrderAddress
OrderStatusHistory
```

Features:

- Order Placement
- Order Tracking
- Status Updates
- Refund Handling
- Cancellation Handling

Services:

```
</> PHP
```



```
OrderService
RefundService
```

Payment Module

Features:

- Stripe
- Paystack
- Webhooks
- Payment Verification
- Transaction Logs

Models:

</> PHP



```
Payment
Transaction
```

Services:

</> PHP



```
PaymentService
TransactionService
```

Notification Module

Implement:

- Email Notifications
- Database Notifications
- Push Notifications
- Flash Sale Launch Alerts
- Queue Entry Alerts
- Back-In-Stock Alerts
- Order Updates

Services:

</> PHP



```
NotificationService
```

Analytics Module

Models:

</> PHP



```
ConversionMetric
DropAnalytics
TrafficAnalytics
```

Reports:

- Conversion Rate
- Revenue Per Drop
- Product Performance
- Inventory Performance
- Customer Retention
- Subscription Metrics

Services:

⌕ PHP



```
AnalyticsService
ReportingService
```

Activity Logs

Tenant application only.

Implement using Spatie Activitylog.

Track:

- Product Changes
- Order Updates
- User Actions
- Inventory Changes
- Subscription Events
- Flash Sale Events

Media Management

Use Spatie Media Library.

Support:

- Product Images
- Brand Logos
- Category Images
- Store Logo
- Store Banner

Create dedicated Media Services.

API Standards

Generate:

- Versioned APIs

```
/api/v1/central/*  
/api/v1/tenant/*
```



- Consistent API response format
- API Resources
- Pagination
- Filtering
- Searching
- Sorting

Response structure:

</> JSON



```
{  
  "success": true,  
  "message": "Operation completed successfully.",  
  "data": {},  
  "meta": {}  
}
```

Database Standards

Requirements:

- Use BIGINT auto-increment IDs only
- Do NOT use UUIDs
- Proper foreign keys
- Proper indexes
- Soft Deletes where appropriate
- Audit columns
- Optimized migration names
- Multi-tenant-safe schema design

Documentation Standards

Generate for every module:

- Migration
- Model
- Controller
- Service
- Form Request
- Resource
- Policy
- Event
- Listener

Include complete PHPDoc blocks for:

- Classes
 - Methods
 - Properties
 - Request fields
 - Resource fields
-

Deliverables

For every module:

1. Database Schema
2. Migration Files
3. Models
4. Form Requests
5. API Resources
6. Controllers
7. Services
8. Policies
9. Events
10. Listeners
11. Routes
12. Permissions
13. Activity Logs
14. Media Collections
15. API Documentation
16. Feature Tests
17. Pest Tests

Build everything following Laravel 13 enterprise standards, scalability, maintainability, security, performance optimization, and SaaS multi-tenancy best practices. Generate production-ready code only.